

RAPPORT PROJET GENIE LOGICIEL

AMRIOUI Ariles
AZMANI Abdelwaheb
BOUAZZAOUI Mohamed-Amine
BOUSLIMANI Nassim
CHAOUKI Abdellahm
GI2-D

SOMMAIRE

- 1. Introduction et Présentation**
- 2. Organisation de l'équipe et Flux de travail**
- 3. Développement du projet : Du MVP à l'application finale**
- 4. Architecture et Diagrammes**
- 5. Problèmes rencontrés et Solutions**
- 6. Résultats obtenus et Conclusion**

1. Introduction et Présentation

Dans un contexte métropolitain dense tel que l'Île-de-France, la gestion des urgences vitales constitue un défi majeur de santé publique. Les délais d'acheminement des services de secours sont critiques : chaque minute gagnée dans la prise en charge d'un patient victime d'un arrêt cardiaque peut augmenter ses chances de survie de 6 à 8 %. Cependant, la saturation chronique de certains établissements hospitaliers et la congestion des réseaux routiers freinent régulièrement l'efficacité des interventions.

Face à ce constat, nous avons développé une application d'aide à la décision basée sur une architecture JavaFX. Ce projet repose sur le couplage de méthodes géométriques avancées, la triangulation de Delaunay et les diagrammes de Voronoï avec l'algorithme de routage Dijkstra. Cette combinaison permet non seulement d'identifier l'hôpital le plus pertinent selon la proximité géographique, mais également de calculer l'itinéraire routier le plus rapide en tenant compte du trafic réel.

Au-delà de l'optimisation technique, ce projet intègre une dimension éthique et légale indispensable dans le secteur médical. L'application assure la confidentialité des dossiers cliniques des patients via un modèle de sécurité RBAC (*Role-Based Access Control*), différenciant les droits d'accès entre les médecins régulateurs et le personnel de terrain. Enfin, conformément aux exigences du RGPD, notre solution garantit une totale transparence en fournissant une explicabilité algorithmique, permettant aux professionnels de santé de comprendre et de justifier chaque décision d'orientation prise par le système.

Notre équipe est composée de 5 membres : quatre ex preing et un étudiant issu d'une CPGE. Né de besoins réels identifiés lors de nos cours de Prototypage, ce projet a été développé avec une attention toute particulière accordée aux risques éthiques.

Ce rapport détaille ainsi la démarche de conception de cette application, l'organisation de notre équipe, ainsi que les solutions architecturales déployées pour répondre aux contraintes complexes d'un projet mêlant géométrie algorithmique, routage et conformité réglementaire.

2. Organisation de l'équipe et flux de travail

Afin d'atteindre les objectifs fixés par le cahier des charges tout en assurant une cohésion technique, nous avons mis en place une organisation collaborative structurée :

- Nous avons utilisé un serveur Discord dédié pour centraliser nos échanges quotidiens et organiser des points de synchronisation hebdomadaires (1 à 2 fois par semaine). Ces réunions nous ont permis d'aligner nos visions sur l'architecture et de lever rapidement les blocages techniques rencontrés
- L'intégralité du projet a été développée sur un dépôt Github commun pour faciliter la collaboration et garder une trace de nos modifications.
- Chaque semaine, nous avons présenté nos avancées à notre tutrice. Ces rendez-vous ont été cruciaux pour valider nos choix d'architecture (notamment lors du pivot vers le MVP console) et pour ajuster notre stratégie de développement en fonction de ses recommandations.

3. Développement du projet: Du MVP à l'application finale

Le jour de début officiel du projet, nous avons d'abord lu le cahier des charges complet afin de prendre connaissance des différentes particularités du projet. Ainsi, nous avons fait un petit résumé des informations les plus importantes sur notre groupe de travail et avons directement contacté notre tutrice pour établir un rendez-vous dans la semaine.

Pendant le laps de temps avant notre rendez-vous, nous avons réfléchi à l'éventuelle répartition des tâches au sein de notre équipe. Ainsi, nous voulions au départ séparer le projet en deux grands axes : Le cœur algorithmique et le cœur graphique. Trois personnes pouvaient s'occuper du code java classique tandis que les deux membres restants s'occuperaient exclusivement de l'interface javafx.

Cependant, après notre première réunion avec la tutrice, nos plans ont dû être modifiés. Il fallait d'abord réaliser un MVP couvrant uniquement la partie console de l'application, c'est pourquoi nous avons passé la première semaine tous ensemble sur la partie algorithmique. Mais avant de se lancer pleinement dans cette tâche, il fallait réaliser les diagrammes d'utilisation et de classes pour partir sur de bonnes bases.

La semaine consacrée au MVP nous a permis de consolider nos connaissances géométriques sur la triangulation de Delaunay et les diagrammes de Voronoï et a permis d'éclaircir nos idées concernant l'application dans sa globalité. Notamment concernant les différentes fonctionnalités auxquelles nous avons réfléchi durant nos cours d'Ethique et de Design.

La semaine qui a suivie a été beaucoup moins chargée car nous étions encore en pleine réflexion sur certains aspects qui ont ralenti notre avancée.

C'est donc à partir de la troisième semaine que nous avons pu commencer pleinement notre véritable projet.

La répartition des tâches était la suivante :

- **Nassim** et **Mohamed-Amine** se sont occupés de l'implémentation Dijkstra et des aspects liés à la sécurité et au RGPD.

- **Ariles** s'est occupé de l'import/export de cartes pour les tests et a aussi servi de lien entre l'équipe algo et l'équipe JavaFx en participant dans les deux groupes.

- **Abdelwaheb** et **Abdellah** se sont occupés de l'interface JavaFx et de la cohérence des modèles géométriques.

Grâce à cette répartition, nous avons pu travailler sereinement sur notre application sans se marcher dessus grâce à la séparation algo/interface de notre code.

4. Architecture et Diagrammes

4.1 Diagramme d'utilisation (voir pdf sur le repository Github)

Le diagramme de cas d'utilisation structure les capacités opérationnelles de l'**Application GPS Médical Intelligent** autour de trois acteurs humains principaux (accès RBAC) et d'un système tiers externe. L'application est segmentée en trois grands modules fonctionnels étanches :

A. Point d'entrée et Sécurité (RBAC)

L'accès à l'ensemble des fonctionnalités est conditionné par le cas d'utilisation «sécurité» Sélectionner son rôle (Combobox RBAC). Cet enregistrement initial permet de restreindre dynamiquement l'interface graphique et de limiter l'accès aux données sensibles selon les droits de l'utilisateur connecté.

B. Module Administration

Ce sous-système est réservé exclusivement à l'acteur **Administrateur (ADMIN)**. Il regroupe les cas d'utilisation liés à la gestion technique de la carte et des données de base :

- **Importer la topologie de la carte** : Chargement de réseaux routiers complexes à partir de fichiers OpenStreetMap (OSM) en format JSON ou de sauvegardes binaires PGLM.
- **Sauvegarder l'infrastructure** : Persistance de la configuration courante du réseau sous format binaire propriétaire (.pglm).
- **Générer des données de simulation** : Injection automatisée de routes, d'hôpitaux et d'incidents aléatoires pour tester la charge et la robustesse des algorithmes.

C. Module Opérationnel

Ce module orchestre la gestion des urgences sur le terrain. Les droits y sont partagés :

- L'**Ambulancier (PARAMEDIC)** et le **Médecin Régulateur (DOCTOR)** coopèrent pour Saisir la localisation et le type d'urgence d'une victime, ou pour Importer des lots d'incidents/sites via fichiers CSV.
- Toute modification ou consultation de la carte déclenche par *inclusion* («include») le cas d'utilisation critique Calculer l'affectation (DispatchEngine).
- Ce cœur algorithmique s'appuie d'une part sur le traitement géométrique (triangulation de Delaunay et diagrammes de Voronoi) et d'autre part sur les données

temps réel fournies par la source externe **Système Trafic** via l'algorithme de calcul d'itinéraire (Dijkstra).

- Le résultat est restitué à travers le cas d'utilisation **Consulter la recommandation et les détails (Map)**, accessible aux personnels de terrain pour visualiser l'hôpital optimal.

D. Module Explicabilité et Éthique (RGPD)

Orienté vers la transparence des décisions automatisées et la protection des données, ce module propose des extensions spécifiques selon le niveau d'accréditation :

- **Consulter le rapport de transparence RGPD** : *En extension («extend»)* de l'affichage de la carte, ce cas d'utilisation permet au **Médecin** et à l'**Administrateur** d'auditer la matrice de scores (facteur de proximité, saturation de l'hôpital, bonus d'historique médical) ayant conduit au choix de l'algorithme.
- **Consulter l'historique du patient** : Également disponible en *extension*, l'accès aux antécédents médicaux est strictement réservé au **Médecin Régulateur (DOCTOR)** pour des raisons évidentes de secret médical.

4.2 Diagramme de classes (Voir pdf sur le repository Github)

Le diagramme de classes de l'application est découpé de manière modulaire en packages, respectant les principes de conception orientée objet et le patron d'architecture MVC (Modèle-Vue-Contrôleur).

A. Package model (Le Cœur Métier)

Ce package regroupe les entités et les structures de données fondamentales de l'application :

- **MapManager** : Classe pivot agissant comme un conteneur centralisé. Elle orchestre et maintient la cohérence entre les éléments géométriques et les entités métiers.
- **Hospital et VictimIncident** : Représentent les entités métiers principales. Hospital (qui hérite de Point) encapsule la capacité d'accueil et les spécialités médicales, tandis que VictimIncident modélise la gravité de l'urgence et l'historique d'un patient.
- **Triangle et VoronoiCell** : Gèrent les structures géométriques abstraites calculées sur les positions des hôpitaux pour optimiser le découpage territorial.
- **RoadNetwork** : Gère le graphe topologique des intersections et des routes pour le calcul d'itinéraires.

B. Package algo (Moteurs de Calcul)

Ce package isole l'intelligence algorithmique de l'application :

- **DispatchEngine** : Calcule en temps réel l'affectation optimale d'un incident vers un hôpital en évaluant une matrice de scores combinée.
- **RoutingEngine** : Calcule les plus courts chemins au sein du RoadNetwork en s'appuyant sur l'algorithme de Dijkstra ou A* combiné aux conditions de trafic.
- **DelaunayEngine et VoronoiEngine** : Génèrent de manière pure (sans bibliothèque tierce) les maillages géométriques à partir des coordonnées des hôpitaux.

C. Package security (Contrôle d'Accès RBAC)

Ce package sécurise l'application en centralisant la gestion des habilitations :

- **UserRole** : Énumération définissant les profils d'accès : ADMIN, DOCTOR (Médecin), et PARAMEDIC (Ambulancier).
- **SecurityContext** : Classe utilitaire globale chargée de mémoriser le rôle actif de la session et d'exécuter les clauses de garde (hasAccess) pour restreindre les actions utilisateur.

D. Package controller (Interface Graphique - MVC)

Ce package pilote l'IHM développée en JavaFX :

- **MainController** : Contrôleur principal qui charge les sous-vues FXML et réalise l'injection de dépendances croisée entre les modules graphiques.
- **SidebarController et MapController** : Fonctionnent en communication bidirectionnelle. MapController gère l'affichage graphique sur la carte interactive, tandis que SidebarController capte les actions utilisateur, interroge le SecurityContext et met à jour les données textuelles.

E. Package explainability (Transparence et Éthique)

Conçu pour répondre aux exigences de transparence (RGPD) :

- **GDPRReportingService** : Analyse la structure DispatchDecision et sa matrice de scores (DecisionScore) pour formaliser un rapport d'audit textuel, explicitant de manière claire la logique de l'affectation automatisée.

F. Package io (Persistance et gestion de fichiers)

- **MapImporterOSM et MapBinarySerializer** : Assurent l'ingestion de données géographiques OpenStreetMap externes ou la sérialisation binaire de l'infrastructure (.pglm).
- **CsvIncidentImporter et CsvSiteImporter** : Réalisent le parsing des fichiers de données plats pour l'intégration en masse d'incidents ou de capacités hospitalières.

5. Problèmes rencontrés et Solutions

La densité fonctionnelle de notre projet nous a confrontés à des défis techniques stimulants, qui nous ont poussés à raffiner continuellement notre approche. Voici quelques exemples :

- **Le défi de l'évolutivité logicielle** : Au fur et à mesure de l'intégration de nouvelles fonctionnalités (Dijkstra, scoring médical, RGPD), notre classe MapManager a progressivement glissé vers le syndrome de la "classe omnisciente" devenant un point de passage trop dense et difficile à maintenir.
 - **Solution** : Nous avons réalisé une refactorisation majeure de notre architecture en appliquant le principe de responsabilité unique (SRP). Nous avons extrait la logique algorithmique vers des classes dédiées (DelaunayEngine, RoutingEngine) et délégué la gestion de la sécurité au SecurityContext.
 - **Résultat** : Le MapManager joue désormais son rôle de véritable chef d'orchestre, orchestrant les services sans saturer son implémentation. Cela a considérablement facilité l'ajout de briques sans impacter la stabilité du moteur géométrique.
- **L'acquisition de données géographiques réelles** : La simulation de l'application sur le territoire de Cergy nécessitait un réseau routier réaliste. Nous avons été confrontés à la difficulté de trouver des données cartographiques exploitables et libres de droits pour simuler un graphe de ville.
 - **Solution** : Après plusieurs tentatives de génération manuelle, nous avons opté pour l'outil **Overpass Turbo**, qui permet d'interroger la base de données collaborative OpenStreetMap. Nous avons extrait les données au format OSM que nous avons ensuite converties pour les rendre compatibles avec la structure de notre graphe (nœuds et arêtes).
 - **Résultat** : Cela nous a permis de valider notre algorithme Dijkstra sur une topologie urbaine réelle et complexe, conférant à notre prototype une crédibilité technique bien supérieure à celle d'une simple grille générée aléatoirement.

- **La gestion de la complexité algorithmique** : Nous avons également dû arbitrer entre la précision des calculs (Dijkstra sur l'intégralité du réseau) et la performance requise pour une application d'urgence.
 - **Solution** : L'utilisation du diagramme de Voronoï comme filtre spatial a été la solution clé. En réduisant le nombre d'hôpitaux candidats à ceux situés dans la zone de garde pertinente, nous avons optimisé le temps de réponse de l'algorithme sans compromettre la pertinence médicale du choix. Nous avons aussi pensé à substituer Dijkstra par A* pour de meilleurs résultats mais finalement ça n'a pas été nécessaire.

6. Résultats obtenus et Conclusion

Le déploiement de cette application nous a permis d'atteindre l'ensemble des objectifs fixés par le cahier des charges, tout en consolidant nos compétences en génie logiciel et en gestion de projet. Les résultats obtenus se déclinent sur trois axes majeurs :

- **Performances techniques et algorithmiques** : Le moteur de calcul, basé sur la triangulation de Delaunay et le routage Dijkstra, remplit pleinement sa fonction. La séparation des responsabilités entre le moteur géométrique et le moteur de routage permet aujourd'hui une réactivité optimale du système. Le routage dynamique, prenant en compte le trafic routier, offre une alternative crédible aux trajets théoriques à vol d'oiseau, augmentant ainsi la précision des temps d'intervention.
- **Responsabilité éthique et conformité RGPD** : La dimension éthique, pilier du projet, a été traitée avec rigueur. Le modèle de sécurité RBAC est opérationnel et garantit le secret médical. De plus, l'intégration du module d'explicabilité permet de transformer une décision algorithmique "opaque" en une justification textuelle claire. Ce résultat répond directement aux enjeux de transparence exigés par le RGPD et renforce la confiance des professionnels de santé dans l'outil.
- **Bilan humain et organisationnel** : Sur le plan humain, ce projet a été une réussite collective. L'utilisation de Git nous a permis d'apprendre à gérer des conflits de code et à synchroniser des développements parallèles. La structure modulaire adoptée en packages a non seulement facilité l'intégration finale entre les différents membres de l'équipe, mais elle rend également notre solution facilement maintenable et extensible pour de futurs développements.

Conclusion : En somme, notre projet ne se résume pas à un simple outil de calcul. Il représente un système cohérent et sécurisé, prêt à être intégré dans un environnement hospitalier réel. Les défis rencontrés, notamment sur l'architecture globale, nous ont permis de développer une approche critique du code, nous orientant vers des pratiques de programmation robustes. Nous sommes fiers d'avoir pu concilier rigueur algorithmique et éthique médicale au sein d'un même écosystème.